

Jagdish Raut

Pune, India | jagdishraut2004@gmail.com | 8767939605 | jagdish.is-a.dev | linkedin.com/in/jagdish2004

Education

Dr. D. Y. Patil College of Engineering, Pune. – Bachelor of Engineering in IT

Nov 2022 – Jul 2026

- CGPA: 8.9/10

Achievements

- **Smart India Hackathon 2024 Grand Finale Winner.**(National) [[Certificate](#)]
 - Global Rank of **269** in TCS CodeVita 2025 among 5.5 lakh+ participants worldwide.
 - Awarded Certificate of Excellence by Infosys at the TechZooka Innovation Challenge 2024. [[Certificate](#)]
 - Ranked **11th** out of 1000+ in CodeRed Competition 2025 (Offline Competitive Programming Contest, Pune). [[Certificate](#)]
 - **Knight** on LeetCode (rating 1861) | **3-Star** on CodeChef (rating 1606) | **800+** problems solved
- [Leetcode](#) [Codechef](#) [GeeksforGeeks](#) github.com/Jagdish2004

Skills / Technologies

Programming Languages: Java, Golang, C++, JavaScript, SQL

Frameworks/ libraries: Spring Boot, Next.js, React.js, Node.js, Express.js, Ent

Databases: PostgreSQL, MongoDB, MySQL, Redis

Tools/ Tech: Unix/Linux, Docker, Git, GitHub, AWS

Experience

Software Engineer Intern, Josh Software

Jan 2026 – June 2026

- Contributed to **KindKart** by building **Golang-based services and GraphQL APIs**, designing **GraphQL and database schemas** for donation workflows, verification systems, and analytics.
- Built a **video processing pipeline** including upload, speech-to-text transcription, and **LLM-based answer extraction**, enabling faster and more transparent verification via admin dashboards.
- Designed an **LLM-driven, component-aware analytics system** that converts user-selected inputs into dynamic queries, selects appropriate frontend visualizations, and integrates with **GraphQL** for real-time chart rendering.
- Created an end-to-end bulk job processing flow involving API calls, successfully managing API rate limits by implementing the **worker pool** pattern using **goroutines**. Processed asynchronous jobs in chunks while maintaining progress tracking.

Projects

Fintech Api: Transaction Risk Detection Engine

Github

- Designed and developed a **monolithic Golang REST API** for transaction risk detection using **vertical slicing architecture**, with **PostgreSQL** for persistence, **Redis** for transaction queuing, and containerized setup using **Docker**.
- Implemented a **score-based fraud detection engine** using statistical metrics such as **Z-score**, **P95 percentile**, **mean**, and **variance** to track user spending behavior and compute **risk scores in O(1) per transaction**.
- Built **background job schedulers** for complex behavioral metrics (not feasible in O(1)) and leveraged **goroutines**, **channels**, and **worker pool concurrency patterns** to parallelize computations and efficiently process high transaction volumes via **Redis queues**, storing results as **user behavior profiles**.
- Implemented **JWT authentication**, developed **rule-based risk scoring** using signals like user behavior, transaction frequency, and device identifiers, and maintained **audit logs** with automated testing.
- Designed a **SOLID-based rule engine** with **dynamic configurable weightage**, enabling extensible detection rules without code changes; automated **database migrations (golang-migrate)**, testing, and build workflows using **Docker Compose** for a reproducible CI-ready environment.

AI FitPro: AI Fitness Tracking Application

Github

- Developed a full stack fitness tracking application with PKCE-based secure authentication, React.js frontend, asynchronous inter-service messaging, fitness monitoring, and context-aware Gemini API to provide personalized fitness recommendations
- Designed a **Java Spring boot Microservice Architecture** featuring multiple services, Eureka Server, API Gateway, centralized configuration, and Keycloak-based OAuth2 authentication; integrated **PostgreSQL** (multi-table) and **MongoDB** (collections) across services with **RabbitMQ** for asynchronous communication.
- Dockerized microservices and deployed them to AWS EC2 using **Docker Compose** for scalable orchestration.